Debugging macOS

Igor Franca_

whoami

> AppSec Specialist & Tech Lead @ `insert_big_bank_here`

> Focado em SigInt e segurança mobile

motivações

whoami

> AppSec Specialist & Tech Lead @ `insert_big_bank_here`

> Focado em SigInt e segurança mobile

> Engenharia de Compiladores, Engenharia Reversa, Game Hacking e Velejar

motivações

whoami

> AppSec Specialist & Tech Lead @ `insert_big_bank_here`

> Focado em SigInt e segurança mobile

> Engenharia de Compiladores, Engenharia Reversa, Game Hacking e Velejar

motivações

> Cansei de ouvir pessoas com medo de debuggers

whoami

> AppSec Specialist & Tech Lead @ `insert_big_bank_here`

> Focado em SigInt e segurança mobile

> Engenharia de Compiladores, Engenharia Reversa, Game Hacking e Velejar

motivações

> Cansei de ouvir pessoas com medo de debuggers

> Ecosistema Apple tem seus quirks, sim

whoami

> AppSec Specialist & Tech Lead @ `insert_big_bank_here`

> Focado em SigInt e segurança mobile

> Engenharia de Compiladores, Engenharia Reversa, Game Hacking e Velejar

motivações

> Cansei de ouvir pessoas com medo de debuggers

> Ecosistema Apple tem seus quirks, sim

> Porque eu acredito lol

mesma instrução, mesmo kernel — o que muda é o sinal no registrador `x16`

	Mach trap	BSD syscall	rotina MIG
exemplo	`task_for_pid()`	`task_read_for_pid()`	`mach_vm_read()`
`x16`	`-45`	`539`	—
entrada	`svc #0x80`	`svc #0x80`	`mach_msg()` → trap `-31`
declarado em	`mach_traps.h`	`syscalls.master`	`.defs` + `mig(1)`

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master` · `osfmk/mach/mach_vm.defs`

`mach_vm.defs` · 627 linhas → 11.058 linhas de C geradas (User.c + Server.c + header)

ta, primeiro, que p***a é um debugger mesmo?

dois processos — e quatro passos quase ofensivos de tão simples



dois processos — e quatro passos quase ofensivos de tão simples



> pegar um task port

no fundo, o plano é esse

dois processos — e quatro passos quase ofensivos de tão simples



> pegar um task port

> criar um jeito de ler e escrever na memória do processo remoto

no fundo, o plano é esse

dois processos — e quatro passos quase ofensivos de tão simples



> pegar um task port

> criar um jeito de ler e escrever na memória do processo remoto

> colocar breakpoints

no fundo, o plano é esse

dois processos — e quatro passos quase ofensivos de tão simples



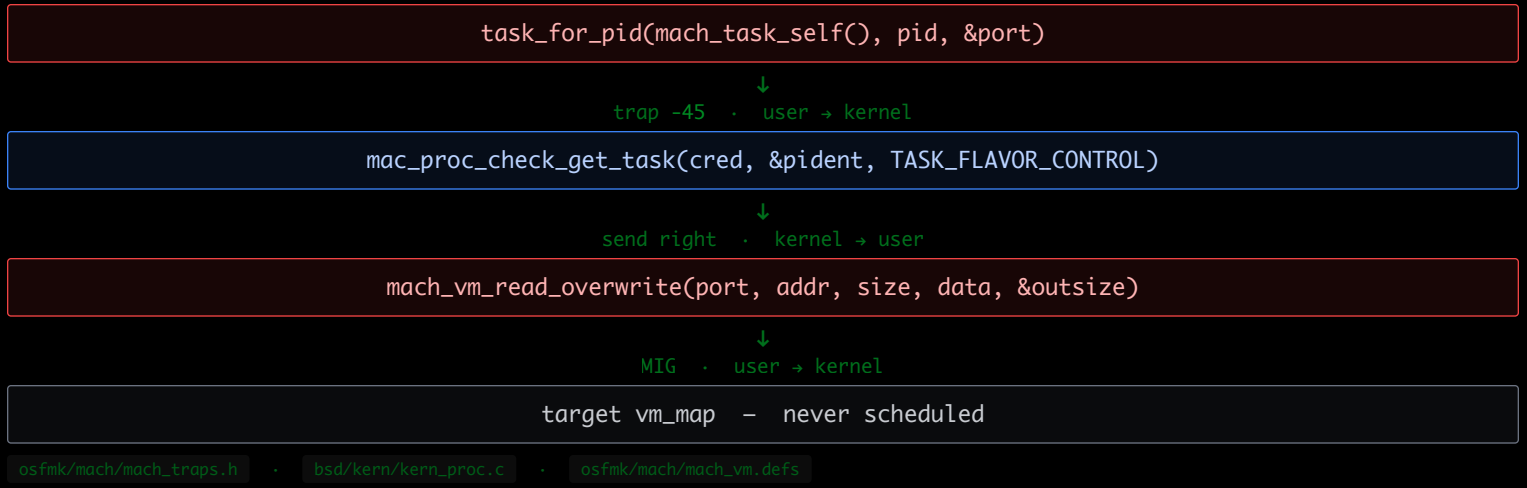
> pegar um task port

> criar um jeito de ler e escrever na memória do processo remoto

> colocar breakpoints

> profit??

mesmo?


task_for_pid(mach_task_self(), pid, &port)
trap -45 · user → kernel
mac_proc_check_get_task(cred, &pident, TASK_FLAVOR_CONTROL)
send right · kernel → user
mach_vm_read_overwrite(port, addr, size, data, &outsize)
MIG · user → kernel
target vm_map — never scheduled
osfmk/mach/mach_traps.h · bsd/kern/kern_proc.c · osfmk/mach/mach_vm.defs

mesmo?

> `DYLD_INTERPOSE` · troca o simbolo antes da `main()` sequer executar, ou seja, nem envolve o kernel

ta, primeiro, que p***a é um debugger mesmo?

> `DYLD_INTERPOSE` · troca o simbolo antes da `main()` sequer executar, ou seja, nem envolve o kernel

> `task_for_pid()` · pede pro kernel a task_port do PID alvo, mas via uma mach trap (id `-45`)

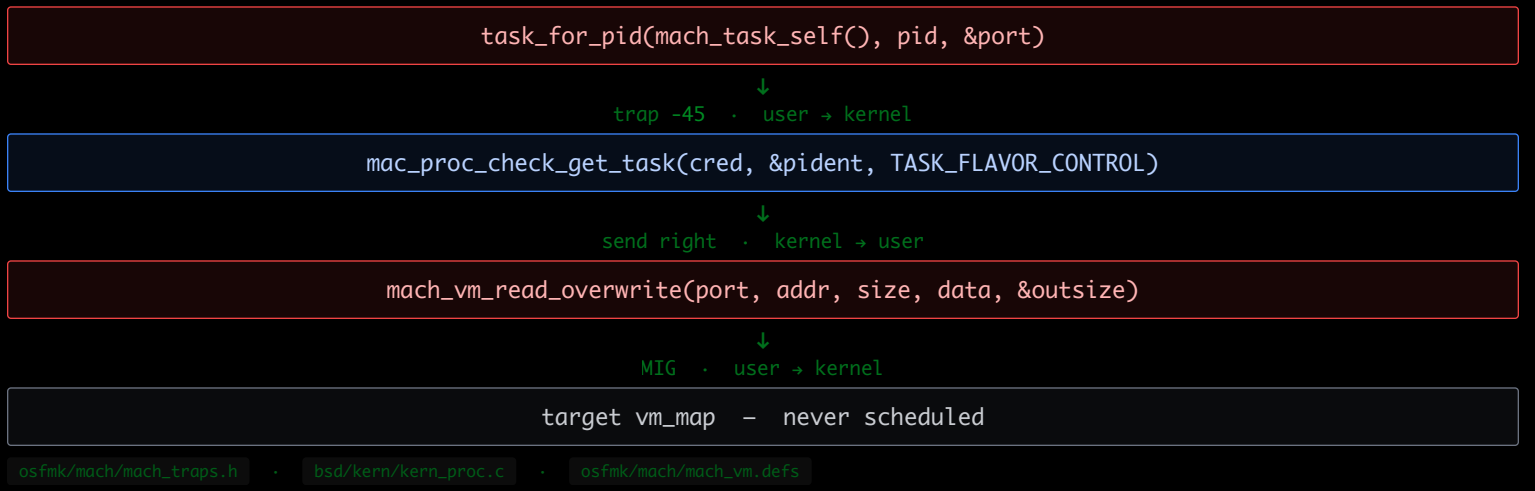
ta, primeiro, que p***a é um debugger mesmo?

> `DYLD_INTERPOSE` · troca o simbolo antes da `main()` sequer executar, ou seja, nem envolve o kernel

> `task_for_pid()` · pede pro kernel a task_port do PID alvo, mas via uma mach trap (id `-45`)

> `task_read_for_pid()` · mesmo esquema, pede pro kernel a task_port, mas via uma syscall (id `539`)

ta, primeiro, que p***a é um debugger mesmo?



```c
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

osfmk/mach/mach_traps.h · bsd/kern/syscalls.master

> Darwin < 9 · `task_for_pid()`, uid-gated

```
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

por baixo dos panos

o "encanamento" Mach + MIG que tudo depende, do Frida ao LLDB

> Darwin < 9 · `task_for_pid()`, uid-gated
>
> - meio que era isso lol

```
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

por baixo dos panos

o "encanamento" Mach + MIG que tudo depende, do Frida ao LLDB

> Darwin < 9 · `task_for_pid()`, uid-gated - meio que era isso lol

> Darwin 9 → 19 · MacOs Leopard adiciona um gate:

```
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

por baixo dos panos

o "encanamento" Mach + MIG que tudo depende, do Frida ao LLDB

> Darwin < 9 · `task_for_pid()`, uid-gated
  - meio que era isso lol
> Darwin 9 → 19 · MacOs Leopard adiciona um gate:
  - usa um daemon chamado `taskgated`, ele é hardenizado pelo SIP

```
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

por baixo dos panos

o "encanamento" Mach + MIG que tudo depende, do Frida ao LLDB

> Darwin < 9 · `task_for_pid()`, uid-gated
  - meio que era isso lol
> Darwin 9 → 19 · MacOs Leopard adiciona um gate:
  - usa um daemon chamado `taskgated`, ele é hardenizado pelo SIP
  - `task_for_pid-allow` · `com.apple.system-task-ports`

```c
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

por baixo dos panos

o "encanamento" Mach + MIG que tudo depende, do Frida ao LLDB

> Darwin < 9 · `task_for_pid()`, uid-gated
  - meio que era isso lol
> Darwin 9 → 19 · MacOs Leopard adiciona um gate:
  - usa um daemon chamado `taskgated`, ele é hardenizado pelo SIP
  - `task_for_pid-allow` · `com.apple.system-task-ports`
> Darwin 20 · Big Sur quebra as task port em portas dedicadas:

```
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

por baixo dos panos

o "encanamento" Mach + MIG que tudo depende, do Frida ao LLDB

> Darwin < 9 · `task_for_pid()`, uid-gated
  - meio que era isso lol
> Darwin 9 → 19 · MacOs Leopard adiciona um gate:
  - usa um daemon chamado `taskgated`, ele é hardenizado pelo SIP
  - `task_for_pid-allow` · `com.apple.system-task-ports`
> Darwin 20 · Big Sur quebra as task port em portas dedicadas:
  - `control > read > inspect > name`

```
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

por baixo dos panos

o "encanamento" Mach + MIG que tudo depende, do Frida ao LLDB

> Darwin < 9 · `task_for_pid()`, uid-gated
  - meio que era isso lol
> Darwin 9 → 19 · MacOs Leopard adiciona um gate:
  - usa um daemon chamado `taskgated`, ele é hardenizado pelo SIP
  - `task_for_pid-allow` · `com.apple.system-task-ports`
> Darwin 20 · Big Sur quebra as task port em portas dedicadas:
  - `control > read > inspect > name`
  - Dropa as Mach traps pra usar syscalls

```
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

por baixo dos panos

o "encanamento" Mach + MIG que tudo depende, do Frida ao LLDB

> Darwin < 9 · `task_for_pid()`, uid-gated
  - meio que era isso lol
> Darwin 9 → 19 · MacOs Leopard adiciona um gate:
  - usa um daemon chamado `taskgated`, ele é hardenizado pelo SIP
  - `task_for_pid-allow` · `com.apple.system-task-ports`
> Darwin 20 · Big Sur quebra as task port em portas dedicadas:
  - `control > read > inspect > name`
  - Dropa as Mach traps pra usar syscalls
> Darwin 21 → ?? · Monterey "granularizou" mais ainda:

```
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

por baixo dos panos

o "encanamento" Mach + MIG que tudo depende, do Frida ao LLDB

> Darwin < 9 · `task_for_pid()`, uid-gated
  - meio que era isso lol
> Darwin 9 → 19 · MacOs Leopard adiciona um gate:
  - usa um daemon chamado `taskgated`, ele é hardenizado pelo SIP
  - `task_for_pid-allow` · `com.apple.system-task-ports`
> Darwin 20 · Big Sur quebra as task port em portas dedicadas:
  - `control > read > inspect > name`
  - Dropa as Mach traps pra usar syscalls
> Darwin 21 → ?? · Monterey "granularizou" mais ainda:
  - `…system-task-ports.control` · `.read`

```c
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

por baixo dos panos

o "encanamento" Mach + MIG que tudo depende, do Frida ao LLDB

> Darwin < 9 · `task_for_pid()`, uid-gated
  - meio que era isso lol
> Darwin 9 → 19 · MacOs Leopard adiciona um gate:
  - usa um daemon chamado `taskgated`, ele é hardenizado pelo SIP
  - `task_for_pid-allow` · `com.apple.system-task-ports`
> Darwin 20 · Big Sur quebra as task port em portas dedicadas:
  - `control > read > inspect > name`
  - Dropa as Mach traps pra usar syscalls
> Darwin 21 → ?? · Monterey "granularizou" mais ainda:
  - `…system-task-ports.control` · `.read`
  - `.name.safe` · `.token.read`

```
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

por baixo dos panos

o "encanamento" Mach + MIG que tudo depende, do Frida ao LLDB

> Darwin < 9 · `task_for_pid()`, uid-gated
  - meio que era isso lol
> Darwin 9 → 19 · MacOs Leopard adiciona um gate:
  - usa um daemon chamado `taskgated`, ele é hardenizado pelo SIP
  - `task_for_pid-allow` · `com.apple.system-task-ports`
> Darwin 20 · Big Sur quebra as task port em portas dedicadas:
  - `control > read > inspect > name`
  - Dropa as Mach traps pra usar syscalls
> Darwin 21 → ?? · Monterey "granularizou" mais ainda:
  - `…system-task-ports.control` · `.read`
  - `.name.safe` · `.token.read`

```
/* mach_traps.h — filed under "Obsolete interfaces." */
kern_return_t task_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

/* syscalls.master — 538 / 539, no public header */
int task_inspect_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);

int task_read_for_pid(
    mach_port_name_t target_tport,
    int pid, mach_port_name_t *t);
```

`osfmk/mach/mach_traps.h` · `bsd/kern/syscalls.master`

talk is cheap

mesma syscall, mesma trap, porém entitlements diferentes

```
libsystem_kernel.dylib`task_for_pid:
  mov  x16, #-0x2d   ; -45, o ID da Mach trap aqui
  svc  #0x80         ; aqui passamos o contexto pro kernel
  ret                ; x0 = the kern_return_t
```

tudo que estamos conversando acontece dentro dessa chamada `svc`.

```
(lldb) finish
(lldb) register read x0

vs target             x0 = 0x0   KERN_SUCCESS
vs target-hardened    x0 = 0x5   KERN_FAILURE
```

Mesma tool, mesma API, mesmo PID. O registrador só serve pra nos mostrar a decisão que o kernel já tomou.

mesma syscall, mesma trap, porém entitlements diferentes

```
libsystem_kernel.dylib`task_for_pid:
  mov  x16, #-0x2d   ; -45, o ID da Mach trap aqui
  svc  #0x80         ; aqui passamos o contexto pro kernel
  ret                ; x0 = the kern_return_t
```

tudo que estamos conversando acontece dentro dessa chamada `svc`.

```
(lldb) finish
(lldb) register read x0

vs target             x0 = 0x0   KERN_SUCCESS
vs target-hardened    x0 = 0x5   KERN_FAILURE
```

Mesma tool, mesma API, mesmo PID. O registrador só serve pra nos mostrar a decisão que o kernel já tomou.

o kernel decide no `exec()`

assinatura + entitlements → AMFI + kernel · `exec()` → csflags

quem lê: o kernel (gate do `task_for_pid`) · o `dyld` (honra ou ignora `DYLD_*`) — só lê, nunca escreve

processo	csflags	bit que decide
target	0x22000205	CS_GET_TASK_ALLOW
target-hardened	0x22011311	CS_RUNTIME · CS_FORCED_LV
launchd	0x26015b11	CS_RESTRICT · CS_PLATFORM_BINARY

osfmk/kern/cs_blobs.h

o kernel decide no `exec()`

assinatura + entitlements → AMFI + kernel · `exec()` → csflags

quem lê: o kernel (gate do `task_for_pid`) · o `dyld` (honra ou ignora `DYLD_*`) — só lê, nunca escreve

processo	csflags	bit que decide
target	0x22000205	CS_GET_TASK_ALLOW
target-hardened	0x22011311	CS_RUNTIME · CS_FORCED_LV
launchd	0x26015b11	CS_RESTRICT · CS_PLATFORM_BINARY

osfmk/kern/cs_blobs.h

bit 0x4 presente → `task_for_pid` passa · ausente → KERN_FAILURE

no fim das contas, é um bit

o kernel decide no `exec()`

`assinatura + entitlements` → `AMFI + kernel · exec()` → `csflags`

quem lê: o kernel (gate do `task_for_pid`) · o `dyld` (honra ou ignora `DYLD_*`) — só lê, nunca escreve

processo	csflags	bit que decide
`target`	`0x22000205`	`CS_GET_TASK_ALLOW`
`target-hardened`	`0x22011311`	`CS_RUNTIME` · `CS_FORCED_LV`
`launchd`	`0x26015b11`	`CS_RESTRICT` · `CS_PLATFORM_BINARY`

`osfmk/kern/cs_blobs.h`

bit **0x4** presente → `task_for_pid` passa · ausente → KERN_FAILURE

bit **0x10** bloqueia `DYLD_INSERT_LIBRARIES` — quem seta é o AMFI, não o `codesign -o runtime`

quem escreve o `csflags` depois do `exec()`

estado do processo	`P_TRACED`	`csflags`	o que entra / sai
intocado	`0`	`0x22000205`	`VALID GET_TASK_ALLOW KILL SIGNED`
só `task_for_pid`	`0`	`0x22000225`	`+INVALID_ALLOWED`
`hddb break` · brk + trap	`0`	`0x22000225`	— nada muda
`lldb` · ou só `ptrace`	`1`	`0x32000025`	`+DEBUGGED` `−KILL`

`./csflags <pid>` · mesmo pid, quatro momentos · nomes sem o prefixo `CS_`

quem escreve o `csflags` depois do `exec()`

estado do processo	P_TRACED	csflags	o que entra / sai
intocado	0	0x22000205	VALID GET_TASK_ALLOW KILL SIGNED
só `task_for_pid`	0	0x22000225	+INVALID_ALLOWED
`hddb break` · brk + trap	0	0x22000225	– nada muda
`lldb` · ou só `ptrace`	1	0x32000025	+DEBUGGED −KILL

`./csflags <pid>` · mesmo pid, quatro momentos · nomes sem o prefixo `CS_`

o task port já custa `+CS_INVALID_ALLOWED` – antes de ler um byte

esse bit muda debaixo de você

quem escreve o `csflags` depois do `exec()`

estado do processo	P_TRACED	csflags	o que entra / sai
intocado	0	0x22000205	VALID GET_TASK_ALLOW KILL SIGNED
só `task_for_pid`	0	0x22000225	+INVALID_ALLOWED
`hddb break` · brk + trap	0	0x22000225	– nada muda
`lldb` · ou só `ptrace`	1	0x32000025	+DEBUGGED −KILL

`./csflags <pid>` · mesmo pid, quatro momentos · nomes sem o prefixo `CS_`

o task port já custa `+CS_INVALID_ALLOWED` – antes de ler um byte

`+CS_DEBUGGED` `-CS_KILL` são assinatura do `ptrace`, não de "ter um debugger"

esse bit muda debaixo de você

quem escreve o `csflags` depois do `exec()`

estado do processo	P_TRACED	csflags	o que entra / sai
intocado	0	0x22000205	VALID GET_TASK_ALLOW KILL SIGNED
só `task_for_pid`	0	0x22000225	+INVALID_ALLOWED
`hddb break` · brk + trap	0	0x22000225	– nada muda
`lldb` · ou só `ptrace`	1	0x32000025	+DEBUGGED –KILL

`./csflags <pid>` · mesmo pid, quatro momentos · nomes sem o prefixo `CS_`

o task port já custa `+CS_INVALID_ALLOWED` – antes de ler um byte

`+CS_DEBUGGED` `-CS_KILL` são assinatura do `ptrace`, não de "ter um debugger"

e `CS_DEBUGGED` não volta atrás – is or HAS BEEN debugged

morde

mesmo comando, mesmo `debugserver` — o alvo é que muda

```
$ lldb -b -o run -- ./target <pass>

p_flag=0x00005806  P_TRACED=1
  0x00000800  P_TRACED
  0x00001000  P_DISABLE_ASLR
csflags=0x32000025
  0x00000020  CS_INVALID_ALLOWED
  0x10000000  CS_DEBUGGED
```

o binário decodifica as duas palavras sozinho — e as duas mudaram.

```
$ lldb -b -o run -- ./target-hardened <pass>

error: attach failed
(Not allowed to attach to process.)
```

o `debugserver` é entitled — e mesmo assim o AMFI recusa.

morde

mesmo comando, mesmo `debugserver` — o alvo é que muda

```
$ lldb -b -o run -- ./target <pass>

p_flag=0x00005806  P_TRACED=1
  0x00000800  P_TRACED
  0x00001000  P_DISABLE_ASLR
csflags=0x32000025
  0x00000020  CS_INVALID_ALLOWED
  0x10000000  CS_DEBUGGED
```

o binário decodifica as duas palavras sozinho — e as duas mudaram.

```
$ lldb -b -o run -- ./target-hardened <pass>

error: attach failed
(Not allowed to attach to process.)
```

o `debugserver` é entitled — e mesmo assim o AMFI recusa.

o alvo não tem entitlement nenhum pra isso — quem decide é o `CS_RUNTIME` gravado no `exec()`

✗ `csops(CS_OPS_SET_STATUS)`

o kernel mascara a entrada: só bits de hardening

```
/* only allow setting a subset
   of all code sign flags */
flags &= CS_HARD | CS_EXEC_SET_HARD |
         CS_KILL | CS_EXEC_SET_KILL |
         CS_RESTRICT | CS_REQUIRE_LV |
         CS_ENFORCEMENT |
         CS_EXEC_SET_ENFORCEMENT;

proc_csflags_set(p, flags);
```

é um OR, nunca um assign — e `CS_GET_TASK_ALLOW` sequer aparece na máscara.

`bsd/kern/kern_proc.c`

tentando atacar essa flag diretamente rola?

quatro tentativas — uma funciona, e uma funciona pela metade

✗ `csops(CS_OPS_SET_STATUS)`

o kernel mascara a entrada: só bits de hardening

✗ `mach_vm_write` na flag

csflags mora no kernel, não no processo

```
$ ./hddb write <pid> <addr> ...

  csflags  →  struct proc, no kernel
  vm_map   →  o que mach_vm_write alcança
```

não tem endereço pra mirar. é um problema "do ovo e da galinha": pra escrever eu precisaria justamente do task port que estou tentando conseguir.

`bsd/sys/proc_internal.h`

tentando atacar essa flag diretamente rola?

quatro tentativas — uma funciona, e uma funciona pela metade

✗ `csops(CS_OPS_SET_STATUS)`

o kernel mascara a entrada: só bits de hardening

✗ `mach_vm_write` na flag

csflags mora no kernel, não no processo

~ patchear o CodeDirectory

limpa `CS_RUNTIME` — volta o `lldb`, mas o `0x4` continua ausente

```
CD flags    0x00010002  →  0x00000002

csflags     0x22011311  →  0x22000201
codesign --verify       →  valid on disk
lldb / DYLD_INSERT      →  voltaram
task_for_pid            →  0x5 ainda
```

um byte flipa `CS_RUNTIME`, `CS_FORCED_LV`, `CS_HARD` e `CS_ENFORCEMENT` — e não adiciona entitlement nenhum.

tentando atacar essa flag diretamente rola?

quatro tentativas — uma funciona, e uma funciona pela metade

✗ `csops(CS_OPS_SET_STATUS)`

o kernel mascara a entrada: só bits de hardening

✗ `mach_vm_write` na flag

csflags mora no kernel, não no processo

~ patchear o CodeDirectory

limpa `CS_RUNTIME` — volta o `lldb`, mas o `0x4` continua ausente

✓ `codesign -s - --entitlements`

reassina; AMFI relê no próximo `exec()`

```
$ codesign -s - -f -o runtime \
    --entitlements get-task-allow.plist th-gta

csflags   0x22010005
          CS_RUNTIME + CS_GET_TASK_ALLOW
task_for_pid            →  kr = 0
```

hardened runtime e debugável ao mesmo tempo. é exatamente um debug build do Xcode.

refiz tudo

mesmo binário, mesma assinatura — só a política do AMFI muda

medição	SIP off	SIP on	o que muda, em nome
`csflags` · `target`	`0x22000005`	`0x22000205`	`+KILL`
`csflags` · `target-hardened`	`0x22010001`	`0x22011311`	`+FORCED_LV +HARD +KILL +ENFORCEMENT`
`csflags` · `launchd`	`0x26014a01`	`0x26015b11`	`+FORCED_LV +HARD +ENFORCEMENT`
`DYLD_INSERT` → `target-hardened`	hook roda	access denied	quem barra: `FORCED_LV`
`lldb` → `target-hardened`	anexa	attach failed	quem barra: `RUNTIME`

nomes sem o prefixo `CS_` · idêntico nos dois: `task_for_pid` nos três alvos · as quatro flavors · o patch no CodeDirectory · `DYLD_INSERT` no `target` · `P_TRACED`

refiz tudo

mesmo binário, mesma assinatura — só a política do AMFI muda

medição	SIP off	SIP on	o que muda, em nome
`csflags` · `target`	`0x22000005`	`0x22000205`	`+KILL`
`csflags` · `target-hardened`	`0x22010001`	`0x22011311`	`+FORCED_LV +HARD +KILL +ENFORCEMENT`
`csflags` · `launchd`	`0x26014a01`	`0x26015b11`	`+FORCED_LV +HARD +ENFORCEMENT`
`DYLD_INSERT` → `target-hardened`	hook roda	access denied	quem barra: `FORCED_LV`
`lldb` → `target-hardened`	anexa	attach failed	quem barra: `RUNTIME`

nomes sem o prefixo `CS_` · idêntico nos dois: `task_for_pid` nos três alvos · as quatro flavors · o patch no CodeDirectory · `DYLD_INSERT` no `target` · `P_TRACED`

a assinatura no disco é byte a byte a mesma — quem muda de ideia é o AMFI, no `exec()`

gente

o objetivo foi sempre o mesmo — ler e escrever na memória de outro processo

tentativa	alvo	resultado	quem barrou
`DYLD_INTERPOSE`	`target`	hook roda antes da `main()`	— nem passa pelo kernel
`DYLD_INTERPOSE`	`target-hardened`	access denied	`CS_FORCED_LV`
`task_for_pid`	`target`	`kr = 0` · read + write	— `get-task-allow`
`task_for_pid`	`target-hardened`	`kr = 5`	AMFI · sem entitlement
`task_read` / `inspect_for_pid`	`target-hardened`	`EPERM` · só `name` sobrevive	AMFI · a flavor não salva
`task_for_pid`	`launchd`	`kr = 5`	AMFI · platform binary
`lldb`	`target-hardened`	attach failed	`CS_RUNTIME`
patch no CodeDirectory	cópia do hardened	`lldb` e `DYLD_INSERT` voltam	o `0x4` continua ausente
`hddb break`	`target`	`brk #0` · `P_TRACED = 0`	—

gente

o objetivo foi sempre o mesmo — ler e escrever na memória de outro processo

tentativa	alvo	resultado	quem barrou
`DYLD_INTERPOSE`	`target`	hook roda antes da `main()`	— nem passa pelo kernel
`DYLD_INTERPOSE`	`target-hardened`	access denied	`CS_FORCED_LV`
`task_for_pid`	`target`	`kr = 0` · read + write	— `get-task-allow`
`task_for_pid`	`target-hardened`	`kr = 5`	AMFI · sem entitlement
`task_read` / `inspect_for_pid`	`target-hardened`	`EPERM` · só `name` sobrevive	AMFI · a flavor não salva
`task_for_pid`	`launchd`	`kr = 5`	AMFI · platform binary
`lldb`	`target-hardened`	attach failed	`CS_RUNTIME`
patch no CodeDirectory	cópia do hardened	`lldb` e `DYLD_INSERT` voltam	o `0x4` continua ausente
`hddb break`	`target`	`brk #0` · `P_TRACED = 0`	—

sobrou uma linha sem bloqueio nenhum — e é ela que vira o nosso debugger

mesmo protocolo, só que a trap acontece do outro lado do cabo



hooka isso e você vê todo o XPC do processo

```
$ dyld_info -exports /usr/lib/system/libxpc.dylib | grep xpc_pipe

    0x00001B9C  _xpc_pipe_create
    0x00001B8C  _xpc_pipe_create_from_port
    0x00033ECC  _xpc_pipe_create_reply_from_port
    0x00005E6C  _xpc_pipe_routine
    0x00033E28  _xpc_pipe_routine_async
    0x00005DDC  _xpc_pipe_routine_with_flags
    0x0001609C  _xpc_pipe_simpleroutine
```

o símbolo é `_xpc_pipe_routine` — com underscore, que é o mangling C de `xpc_pipe_routine()`

17 símbolos `xpc_pipe*` ao todo · e nenhuma declaração: nem em `/usr/include/xpc/`, nem no `libxpc.tbd` do SDK ;)

hooka isso e você vê todo o XPC do processo

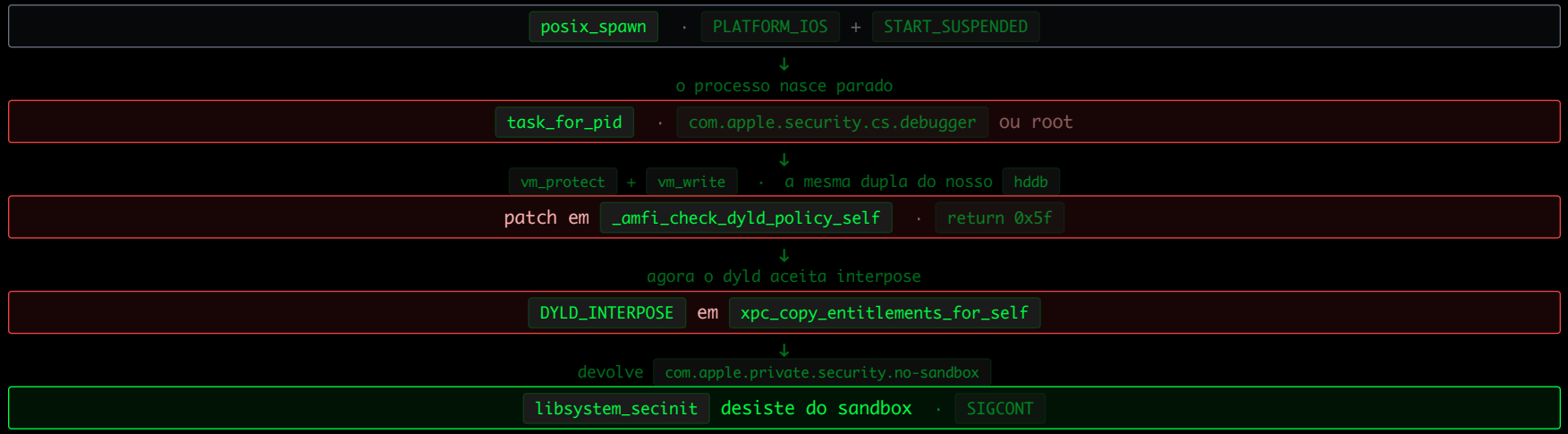
```
$ dyld_info -exports /usr/lib/system/libxpc.dylib | grep xpc_pipe

    0x00001B9C  _xpc_pipe_create
    0x00001B8C  _xpc_pipe_create_from_port
    0x00033ECC  _xpc_pipe_create_reply_from_port
    0x00005E6C  _xpc_pipe_routine
    0x00033E28  _xpc_pipe_routine_async
    0x00005DDC  _xpc_pipe_routine_with_flags
    0x0001609C  _xpc_pipe_simpleroutine
```

o símbolo é `_xpc_pipe_routine` — com underscore, que é o mangling C de `xpc_pipe_routine()`

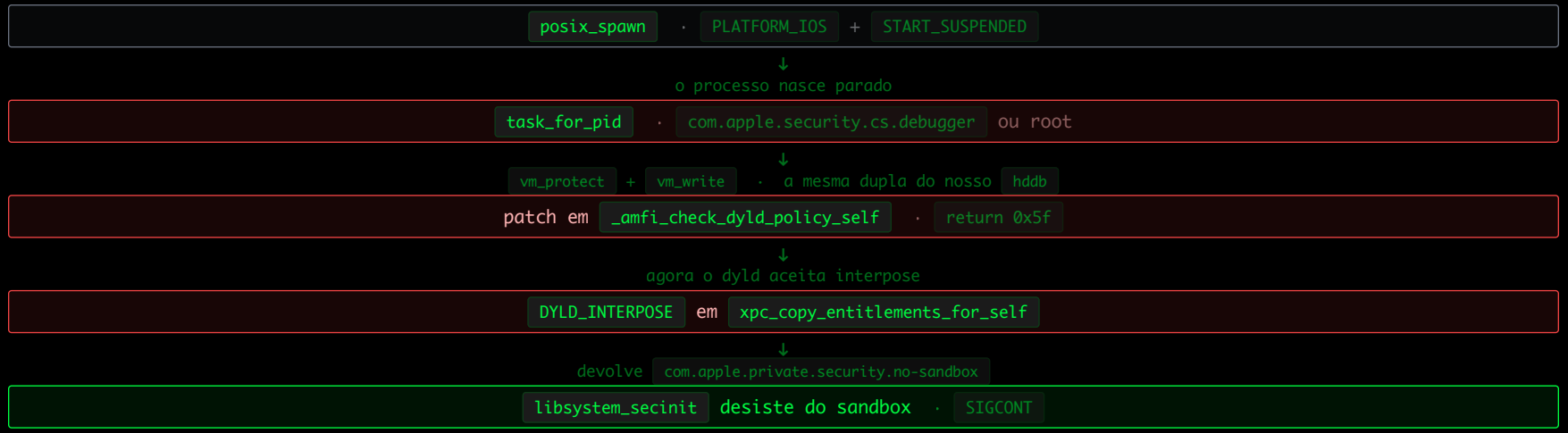
17 símbolos `xpc_pipe*` ao todo · e nenhuma declaração: nem em `/usr/include/xpc/`, nem no `libxpc.tbd` do SDK ;)

rodar código iOS nativo no Mac — e por que isso é essa talk inteira



Samuel Groß · Project Zero · 2021 — projectzero.google/2021/05/fuzzing-ios-code-on-macos-at-native.html

rodar código iOS nativo no Mac — e por que isso é essa talk inteira

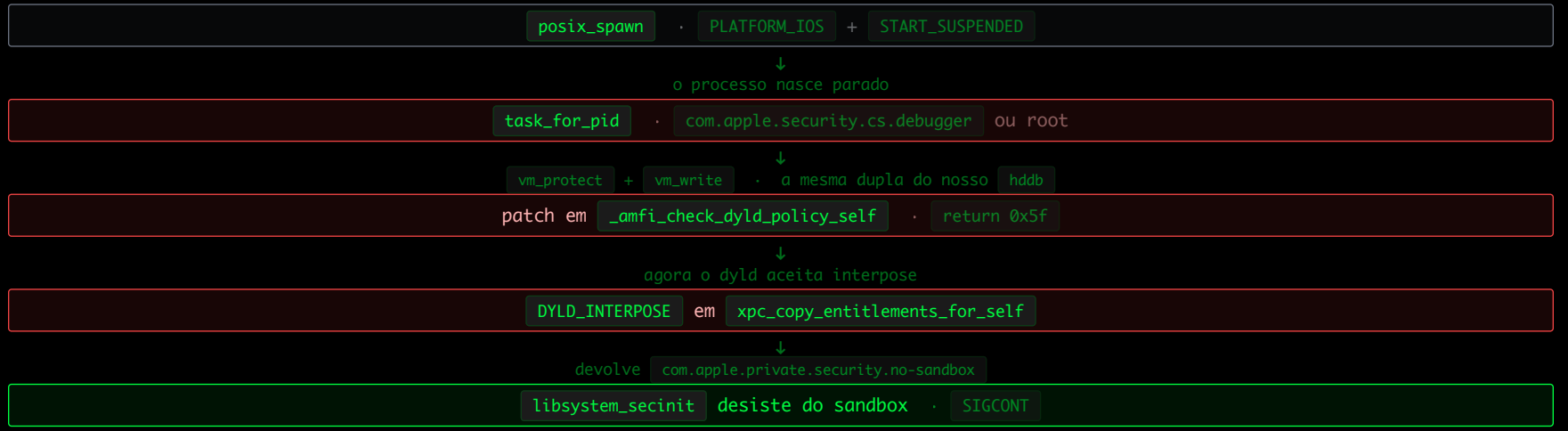


sem isso: `secinit` pede perfil pro `secinitd(8)`, que não reconhece o binário iOS, e o processo dá `abort(3)` antes da `main()`.

Samuel Groß · Project Zero · 2021 — projectzero.google/2021/05/fuzzing-ios-code-on-macos-at-native.html

uns macetes aleatórios #2

rodar código iOS nativo no Mac — e por que isso é essa talk inteira



sem isso: `secinit` pede perfil pro `secinitd(8)`, que não reconhece o binário iOS, e o processo dá `abort(3)` antes da `main()`.

`secinit` não é hardened runtime: userspace antes da `main()`, decide sandbox — enquanto `CS_RUNTIME` é kernel, no `exec()`. mesmo blob de entitlements, dois momentos.

Samuel Groß · Project Zero · 2021 — projectzero.google/2021/05/fuzzing-ios-code-on-macos-at-native.html

obrigado!

Igor Franca

obrigado especial a @mobseccrew
e a @appleredteamvillage
e a vocês por assistirem



github.com/Horaddrim



linkedin.com/in/igor.franca